

geo

All pair Segment intersection

```
const ld EPS = 1e-11;

struct Point {
    ld x, y;
};

struct Segment {
    Point p, q;
    int id;

    ld get_y(ld x) const {
        if (abs(p.x - q.x) < EPS) return p.y;
        return p.y + (q.y - p.y) * (x - p.x) / (q.x - p.x);
    }
};

ld sweep_x;

bool operator<(const Segment &a, const Segment &b) {
    if (a.id == b.id) return false;
    ld ya = a.get_y(sweep_x);
    ld yb = b.get_y(sweep_x);
    if (abs(ya - yb) > EPS) return ya < yb;
    return a.id < b.id;
}

ld cross_product(Point a, Point b, Point c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
}

bool on_segment(Point p, Segment s) {
    return p.x >= min(s.p.x, s.q.x) - EPS && p.x <= max(s.p.x, s.q.x) + EPS &&
        p.y >= min(s.p.y, s.q.y) - EPS && p.y <= max(s.p.y, s.q.y) + EPS;
}

bool intersect(Segment s1, Segment s2) {
    ld d1 = cross_product(s1.p, s1.q, s2.p);
    ld d2 = cross_product(s1.p, s1.q, s2.q);
    ld d3 = cross_product(s2.p, s2.q, s1.p);
    ld d4 = cross_product(s2.p, s2.q, s1.q);
```

```

    if (((d1 > EPS && d2 < -EPS) || (d1 < -EPS && d2 > EPS)) &&
        ((d3 > EPS && d4 < -EPS) || (d3 < -EPS && d4 > EPS)))
        return true;

    if (abs(d1) < EPS && on_segment(s2.p, s1)) return true;
    if (abs(d2) < EPS && on_segment(s2.q, s1)) return true;
    if (abs(d3) < EPS && on_segment(s1.p, s2)) return true;
    if (abs(d4) < EPS && on_segment(s1.q, s2)) return true;

    return false;
}

struct Event {
    ld x;
    int type;
    int id;

    bool operator<(const Event &other) const {
        if (abs(x - other.x) > EPS) return x < other.x;
        return type > other.type;
    }
};

bool has_intersection(vector<Segment> &segments) {
    int n = segments.size();
    vector<Event> events;
    for (int i = 0; i < n; i++) {
        if (segments[i].p.x > segments[i].q.x) swap(segments[i].p,
segments[i].q);
        events.push_back({segments[i].p.x, 1, i});
        events.push_back({segments[i].q.x, -1, i});
    }
    sort(events.begin(), events.end());

    set<Segment> active;
    vector<set<Segment>::iterator> iters(n);

    for (auto &e: events) {
        sweep_x = e.x;
        if (e.type == 1) {
            auto it = active.lower_bound(segments[e.id]);
            if (it != active.end() && intersect(segments[e.id], *it)) return
true;
            if (it != active.begin() && intersect(segments[e.id], *prev(it)))
return true;
        }
    }
}

```

```

        iters[e.id] = active.insert(it, segments[e.id]);
    } else {
        auto it = iters[e.id];
        auto nxt = next(it), prv = (it == active.begin() ? active.end() :
prev(it));
        if (nxt != active.end() && prv != active.end() && intersect(*nxt,
*prv)) return true;
        active.erase(it);
    }
}
return false;
}

```

Basics

```

const int inf = 1e18;
const ld EPS = 1e-9;
const ld pi = acos(-1);
typedef ld T;
typedef complex<T> pt;
#define X real()
#define Y imag()

```

```

// comparison
bool lt(ld a, ld b) { return a < b - eps; }
bool gt(ld a, ld b) { return lt(b, a); }
bool lteq(ld a, ld b) { return !lt(b, a); }
bool gteq(ld a, ld b) { return !lt(a, b); }
bool eq(ld a, ld b) { return abs(a - b) <= eps; }

```

```

bool cmp(const pt &a, const pt &b) { return make_pair(a.X, a.Y) <
make_pair(b.X, b.Y); }
T sq(pt p) { return p.X * p.X + p.Y * p.Y; }
T dot(pt v, pt w) { return v.X * w.X + v.Y * w.Y; }
T cross(pt v, pt w) { return v.X * w.Y - v.Y * w.X; }
ld dist(pt a, pt b) { return sqrtl(sq(a - b)); }
int sgn(T val) { return (val > EPS) - (val < -EPS); }
bool isPerp(pt v, pt w) { return fabs(dot(v, w)) < EPS; }
pt perp(pt p) { return {-p.Y, p.X}; }
ld toDeg(ld ang) { return ang * 180.0 / pi; }
ld toRad(ld ang) { return ang * pi / 180.0; }
bool same(pt a, pt b) { return !sgn(a.X - b.X) and !sgn(a.Y - b.Y); }

```

angles

```
//positive for CCW, negative for CW
T orient(pt a, pt b, pt c) { return cross(b - a, c - a); }
T clamp(T val, T low, T high) { return max(low, min(val, high)); }
ld angle(pt v, pt w) { return acos(clamp(dot(v, w) / abs(v) / abs(w), (T) - 1.0, (T) 1.0)); }
// x,y adjacent sides | z opposite side
ld angle(T x, T y, T z) { return acos(clamp((x * x + y * y - z * z) / (2 * x * y), (T) - 1.0, (T) 1.0)); }

T orientedAngle(pt a, pt b, pt c) {
    ld ampli = angle(b - a, c - a);
    if (orient(a, b, c) > 0)
        return ampli;
    else
        return 2 * pi - ampli;
}

T angleTravelled(pt a, pt b, pt c) {
    ld ampli = angle(b - a, c - a);
    if (orient(a, b, c) > 0)
        return ampli;
    else
        return -ampli;
}

// check p in between angle(bac) counter clockwise
bool inAngle(pt a, pt b, pt c, pt p) {
    T abp = orient(a, b, p), acp = orient(a, c, p), abc = orient(a, b, c);
    if (abc < 0)
        swap(abp, acp);
    return (abp >= 0 && acp <= 0) ^ (abc < 0);
}
```

Circle

```
struct Circle
{
    pt o;
    T r;

    Circle() {}
    Circle(pt o, T r) : o(o), r(r) {}
}
```

```

Circle(pt a, pt b, pt c)
{
    b -= a, c -= a;
    assert(cross(b, c) != 0);
    pt center = a + perp(b * sq(c) - c * sq(b)) / cross(b, c) / (T)2;
    T radius = abs(center - a);
    r = radius;
    o = center;
}

// Line intersection: returns number of intersection points, stores them in
'out'
int circleLine(Line l, pair<pt, pt> &out) const
{
    T h2 = r * r - l.sqDist(o);
    if (h2 >= 0)
    {
        pt p = l.proj(o);
        pt h = l.v * (T)(sqrt(h2) / abs(l.v));
        out = {p - h, p + h};
    }
    return 1 + sgn(h2);
}

// Circle-circle intersection: returns number of intersection points, stores
them in 'out'
int circleCircle(pt o1, T r1, pt o2, T r2, pair<pt, pt> &out)
{
    pt d = o2 - o1;
    T d2 = sq(d);
    if (d2 == 0)
    {
        assert(r1 != r2);
        return 0;
    } // concentric circles
    T pd = (d2 + r1 * r1 - r2 * r2) / 2; // = |O_1P| * d
    T h2 = r1 * r1 - pd * pd / d2; // = h^2
    if (h2 >= 0)
    {
        pt p = o1 + d * pd / d2, h = perp(d) * (T)sqrt(h2 / d2);
        out = {p - h, p + h};
    }
    return 1 + sgn(h2);
}

```

```

// Tangents from this to another circle: returns number of tangent lines,
fills 'out'
int tangents(const Circle &c2, bool inner, vector<pair<pt, pt>> &out) const
{
    T r2 = inner ? -c2.r : c2.r;
    pt d = c2.o - o;
    T dr = r - r2, d2 = sq(d), h2 = d2 - dr * dr;
    if (d2 == 0 || h2 < 0)
    {
        assert(h2 != 0);
        return 0;
    }
    for (T sign : {-1, 1})
    {
        pt v = (d * dr + perp(d) * (T)sqrt(h2) * sign) / d2;
        out.push_back({o + v * r, c2.o + v * r2});
    }
    return 1 + (h2 > 0);
}

// Area of intersection with another circle
ld intersection_area(const Circle &c) const
{
    ld dx = o.X - c.o.X;
    ld dy = o.Y - c.o.Y;
    ld d = sqrtl(dx * dx + dy * dy);

    if (d >= r + c.r)
        return 0.0L;
    if (d + c.r <= r)
        return pi * c.r * c.r;
    if (d + r <= c.r)
        return pi * r * r;

    // angle and segment area for c
    ld ct = (c.r * c.r + d * d - r * r) / (2.0L * c.r * d);
    ld theta = acosl(ct);
    ld area1 = c.r * c.r * (theta - sinl(theta) * cosl(theta));

    // angle and segment area for this circle
    ct = (r * r + d * d - c.r * c.r) / (2.0L * r * d);
    theta = acosl(ct);
    ld area2 = r * r * (theta - sinl(theta) * cosl(theta));

    return area1 + area2;
}

```

```

    bool operator==(const Circle &c) const { return c.o == o and sgn(c.r - r) ==
0 and sgn(r) > 0; }
};

```

```

struct Circle {
    point center;
    ld r;

    Circle() {
    }

    Circle(point _o, ld _r) : center(_o), r(_r) {
    }

    Circle(point a, point b, point c) {
        point m1 = (b + a) * 0.5l, v1 = b - a;
        point pv1 = prep(v1);
        point m2 = (b + c) * 0.5l, v2 = b - c;
        point pv2 = prep(v2);
        point end1 = m1 + pv1, end2 = m2 + pv2, cen;
        intersectll(m1, end1, m2, end2, cen, 15);
        this->center = cen;
        this->r = abs(a - center);
    }

    bool operator ==(Circle v) { return center == v.center && sign(r - v.r) ==
0; }

    double area() { return pi * r * r; }
    double circumference() { return 2.0 * pi * r; }
};

// 0 -> outside , 1 -> on circumference , 2-> inside
int Circlepointrel(Circle cir, point a) {
    ld d = dist(cir.center - a);
    if (sign(d - cir.r) < 0) return 2;
    if (sign(d - cir.r) == 0) return 1;
    return 0;
}

int Circlelinerel(Circle cir, line l) {
    ld d = l.dist(cir.center);
    if (sign(d - cir.r) < 0) return 2;
    if (sign(d - cir.r) == 0) return 1;
    return 0;
}

```

```

vector<point > CircleLineInter(Circle cir, line l) {
    ld d = l.dist(cir.center);
    if (gt(d, cir.r)) return {};
    point p = l.proj(cir.center);
    if (eq(d, cir.r)) return {p};
    ld h = sqrt(cir.r * cir.r - d * d);
    point dir = prep(l.n);
    dir = normalize(dir) * h;
    return {p + dir, p - dir};
}

// return:
// 0 -> no intersect (separated)
// 1 -> external tangent
// 2 -> two intersection points
// 3 -> internal tangent
// 4 -> one inside another (disjoint)
// 5 -> identical circles
int cirCirRel(Circle A, Circle B) {
    ld d = abs(A.center - B.center);
    ld r1 = A.r, r2 = B.r;
    if (same(A.center, B.center) and eq(r1, r2)) return 5;
    if (gt(d, r1 + r2)) return 0;
    if (lt(d, abs(r1 - r2))) return 4;
    if (eq(d, r1 + r2)) return 1;
    if (eq(d, abs(r1 - r2))) return 3;
    return 2;
}

vector<point > CircleCircleInter(Circle A, Circle B) {
    point c1 = A.center, c2 = B.center;
    ld r1 = A.r, r2 = B.r;
    ld d = abs(c1 - c2);
    if (same(c1, c2) and eq(r1, r2) && gt(r1, 0)) return vector<point >(3, c1);
    if (gt(d, r1 + r2)) return {};
    if (lt(d, abs(r1 - r2))) return {};
    ld ang1 = angvecx(c2 - c1);
    ld ang2 = getAngle_A_abc(r2, r1, d);
    if (isnan(ang2)) ang2 = 0;
    point p1 = polar(r1, ang1 + ang2) + c1;
    if (eq(dot(p1 - c1, p1 - c1), r1 * r1) == false or eq(dot(p1 - c2, p1 -
c2), r2 * r2) == false) return {};
    point p2 = polar(r1, ang1 - ang2) + c1;
    vector<point > ans;
    ans.push_back(p1);

```

```

        if (!same(p1, p2)) ans.push_back(p2);

    return ans;
}

Circle minimum_enclosing_circle(vector<point > p) {
    static std::mt19937 rng((unsigned)
chrono::steady_clock::now().time_since_epoch().count());
    shuffle(p.begin(), p.end(), rng);
    int n = (int) p.size();
    Circle c(point(0, 0), 0);
    if (n == 0) return c;
    c = Circle(p[0], 0);
    for (int i = 1; i < n; ++i) {
        if (sign(abs(c.center - p[i]) - c.r) > 0) {
            c = Circle(p[i], 0);
            for (int j = 0; j < i; ++j) {
                if (sign(abs(c.center - p[j]) - c.r) > 0) {
                    point cen = (p[i] + p[j]) / 2.0L;
                    c = Circle(cen, abs(p[i] - cen));
                    for (int k = 0; k < j; ++k) {
                        if (sign(abs(c.center - p[k]) - c.r) > 0) {
                            c = Circle(p[i], p[j], p[k]);
                        }
                    }
                }
            }
        }
    }
    return c;
}

```

Closest pair problem

```

struct Point {
    int x, y, id;
};

int dist(Point a, Point b) {
    int dx = a.x - b.x;
    int dy = a.y - b.y;
    return dx * dx + dy * dy;
}

```

```

void solve(int tc) {
    int n;
    cin >> n;
    vector<Point> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i].x >> a[i].y;
        a[i].id = i;
    }
    sort(a.begin(), a.end(), [](const Point& l, const Point& r) {
        if (l.x != r.x) return l.x < r.x;
        return l.y < r.y;
    });
    set<pair<int, int>> st;
    int ans = dist(a[0], a[1]); a[ifor]
    pair<int, int> indices = {a[0].id, a[1].id};
    int left = 0;
    for (int i = 0; i < n; i++) {
        int d = ceil(sqrt(ans));
        while (left < i and (a[i].x - a[left].x) * (a[i].x - a[left].x) >=
ans) {
            st.erase({a[left].y, left});
            left++;
        }
        auto it1 = st.lower_bound({a[i].y - d, -1});
        auto it2 = st.upper_bound({a[i].y + d, n});
        for (auto it = it1; it != it2; ++it) {
            int j = it->second;
            int cur = dist(a[i], a[j]);
            if (cur < ans) {
                ans = cur;
                indices = {a[i].id, a[j].id};
            }
        }
        st.insert({a[i].y, i});
    }
    cout << min(indices.first, indices.second) << " "
        << max(indices.first, indices.second) << " "
        << fixed << setprecision(6) << sqrt(ans) << endl;
}

```

```

struct Point {
    ld x, y;
};

ld dist(Point a, Point b) {

```

```

    ld dx = a.x - b.x;
    ld dy = a.y - b.y;
    return dx * dx + dy * dy;
}

void solve() {
    while (true) {
        int n;
        cin >> n;

        if (n == 0) return;

        vector<Point> a(n);

        for (int i = 0; i < n; i++)
            cin >> a[i].x >> a[i].y;

        sort(a.begin(), a.end(), [](auto &l, auto &r) {
            if (l.x != r.x) return l.x < r.x;
            return l.y < r.y;
        });

        set<pair<ld, int> > st;

        ld ans = dist(a[0], a[1]);
        pair<int, int> best = {0, 1};

        int left = 0;

        for (int i = 0; i < n; i++) {
            ld d = sqrt(ans);

            while (left < i && (a[i].x - a[left].x) > d) {
                st.erase({a[left].y, left});
                left++;
            }

            auto it1 = st.lower_bound({a[i].y - d, -1});
            auto it2 = st.upper_bound({a[i].y + d, n});

            for (auto it = it1; it != it2; ++it) {
                int j = it->second;

                ld cur = dist(a[i], a[j]);

                if (cur < ans) {

```

```

        ans = cur;
        best = {i, j};
    }
}

st.insert({a[i].y, i});
}

auto print = [&](Point p) {
    cout << fixed << setprecision(2) << p.x << " " << p.y << " ";
};

if (best.first > best.second)
    swap(best.first, best.second);

print(a[best.first]);
print(a[best.second]);
cout << endl;
}
}

```

Convex Hull

```

vector<pt> lower_hull(vector<pt> &points) {
    // make sure points are sorted with cmp
    vector<pt> lower;
    for (auto &p: points) {
        while (lower.size() > 1 && orient(lower[lower.size() - 2],
lower.back(), p) <= 0) // < to include collinear
        {
            lower.pop_back();
        }
        lower.push_back(p);
    }
    return lower;
}

vector<pt> upper_hull(vector<pt> &points) {
    // make sure points are sorted with cmp
    vector<pt> upper;
    for (auto &p: points) {
        while (upper.size() > 1 && orient(upper[upper.size() - 2],
upper.back(), p) >= 0) // > to include collinear
        {
            upper.pop_back();
        }
    }
    return upper;
}

```

```

        }
        upper.push_back(p);
    }
    return upper;
}

vector<pt> convex_hull(vector<pt> &p) {
    if (p.size() <= 1)
        return p;

    vector<pt> sorted = p;
    sort(all(sorted), cmp);

    vector<pt> lower = lower_hull(sorted);
    vector<pt> upper = upper_hull(sorted);

    // Remove duplicate endpoints
    lower.pop_back();
    reverse(upper.begin(), upper.end());
    upper.pop_back();

    // Combine
    lower.insert(lower.end(), upper.begin(), upper.end());

    if (lower.size() == 2 && lower[0] == lower[1])
        lower.pop_back();

    return lower;
}

```

other implementation

```

struct Line {
    point a, b;
};

ld dist(point a, point b) {
    return abs(cross(b - a, a)) / abs(b - a);
}

bool ccw(point a, point b, point c) {
    return cross(b - a, c - b) > 0;
}

void convex_hull(vector<point > &a, vector<point > &h) {

```

```

    sort(a.begin(), a.end(), [](point p1, point p2) {
        if (!eq(real(p1), real(p2))) return real(p1) < real(p2);
        return imag(p1) < imag(p2);
    });
    vector<point> stk;
    for (auto &p: a) {
        while (stk.size() >= 2 and !ccw(stk[stk.size() - 2], stk[stk.size() -
1], p)) stk.pop_back();
        stk.push_back(p);
    }
    int tmp = stk.size();
    for (int i = (int) a.size() - 2; i >= 0; i--) {
        point p = a[i];
        while (stk.size() > tmp and !ccw(stk[stk.size() - 2], stk[stk.size() -
1], p)) stk.pop_back();
        stk.push_back(p);
    }
    h = stk;
}

```

Formulas

Heron

$$= \frac{a+b+c}{2} \sqrt{\frac{(a+b+c)(-a+b+c)(a-b+c)(a+b-c)}{4}}$$

$$= \sqrt{s(s-a)(s-b)(s-c)}$$

a, b, c there are the lengths of the sides

Law of sines and cosines

$$\frac{a}{\sin(A)} = \frac{b}{\sin(B)} = \frac{c}{\sin(C)} = d$$

$$c^2 = a^2 + b^2 - 2ab \cos(C)$$

a, b, c : lengths, A, B, C : opposite angles, d : circumcircle

Helper Functions

```

vector<pt> remove_colinear(vector<pt> pts)
{
    vector<pt> use;
    use.push_back(pts[0]);
    use.push_back(pts[1]);
    for (int i = 2; true; ++i)
    {

```

```

    pt a = use[(int)use.size() - 2];
    pt b = use[(int)use.size() - 1];
    if (sgn(cross(b - a, pts[i % (int)pts.size()] - a)) == 0)
        use.pop_back();
    if (i == (int)pts.size())
        break;
    use.push_back(pts[i]);
}
pt a = use[(int)use.size() - 1];
pt b = use[0];
pt c = use[1];
if (sgn(cross(b - a, c - a)) == 0)
    use.erase(use.begin());
return use;
}

void prepareConvexPolygon(vector<pt> &v)
{
    // 1. Find the point with the lowest y (break ties by x)
    int pivot = 0;
    for (int i = 1; i < v.size(); ++i)
        if (v[i].Y < v[pivot].Y || (v[i].Y == v[pivot].Y && v[i].X < v[pivot].X))
            pivot = i;

    // 2. Rotate the polygon so that pivot is at index 0
    rotate(v.begin(), v.begin() + pivot, v.end());

    // 3. Ensure polygon is CCW
    if (orient(v[0], v[1], v[2]) < 0)
        reverse(v.begin() + 1, v.end()); // Keep v[0], reverse rest
}

// rotate the polygon such that the (bottom, left)-most point is at the first
// position
void reorder_polygon(vector<pt> &p)
{
    int pos = 0;
    for (int i = 1; i < p.size(); i++)
    {
        if (p[i].Y < p[pos].Y || (sgn(p[i].Y - p[pos].Y) == 0 && p[i].X <
p[pos].X))
            pos = i;
    }
    rotate(p.begin(), p.begin() + pos, p.end());
}

```

```

bool point_in_triangle(pt a, pt b, pt c, pt p, bool strictly_in = false)
{
    int sign1 = cross(b - a, p - a);
    int sign2 = cross(c - b, p - b);
    int sign3 = cross(a - c, p - c);
    if (strictly_in)
        return ((sign1 > 0 and sign2 > 0 and sign3 > 0) or (sign1 < 0 and sign2 <
0 and sign3 < 0));
    else
        return ((sign1 >= 0 and sign2 >= 0 and sign3 >= 0) or (sign1 <= 0 and
sign2 <= 0 and sign3 <= 0));
}

bool half(pt p) // true if in blue half
{
    assert(p.X != 0 || p.Y != 0); // the argument of (0,0) is undefined
    return p.Y > 0 || (p.Y == 0 && p.X < 0);
}

void polarSort(vector<pt> &v)
{
    sort(v.begin(), v.end(), [](pt v, pt w)
        { return make_tuple(half(v), 0) < make_tuple(half(w), cross(v, w)); });
}

void sort_cw(vector<point > &p) {
    int n = p.size();
    point ctr = {0, 0};
    for (auto &x: p) ctr += x;
    ctr /= (ld) n;

    sort(p.begin(), p.end(), [&](const point &a, const point &b) {
        ld A = atan2(a.Y - ctr.Y, a.X - ctr.X);
        ld B = atan2(b.Y - ctr.Y, b.X - ctr.X);
        return A > B;
    });
}

void sort_ccw(vector<point > &p) {
    sort_cw(p);
    reverse(p.begin(), p.end());
}

```

Line

```

struct Line {
    pt v;
    T c;

    Line(pt v, T c) : v(v), c(c) {
    }

    // from equation ax+by = c
    Line(T a, T b, T _c) {
        v = {b, -a};
        c = _c;
    }

    // line from two points
    Line(pt p, pt q) {
        v = q - p;
        c = cross(v, p);
    }

    T side(pt p) { return cross(v, p) - c; } // test point location with line
0 if on the line
    ld dist(pt p) { return abs(side(p)) / abs(v); }
    double sqDist(pt p) { return side(p) * side(p) / (T) sq(v); }
    //new line perpendicular to this one passing through p
    Line perpThrought(pt p) { return {p, p + perp(v)}; }

    bool cmpProj(pt p, pt q) {
        return dot(v, p) < dot(v, q);
    }
    // Moves the line by displacement vector t
    Line translate(pt t) { return {v, c + cross(v, t)}; }
    Line shiftLeft(T dist) { return {v, c + dist * abs(v)}; } // shift right
by negative dist
    // Returns the point on the line closest to p (orthogonal projection)
    pt proj(pt p) { return p - perp(v) * side(p) / sq(v); }
    pt refl(pt p) { return p - perp(v) * (T) 2.0 * side(p) / sq(v); }
};

bool inter(Line l1, Line l2, pt &out) // out is the intersection point
{
    T d = cross(l1.v, l2.v);
    if (fabs(d) < EPS)
        return false;
    out = (l2.v * l1.c - l1.v * l2.c) / d; // requires floating-point
coordinates
    return true;
}

```

```

}
// Returns the angle bisector between two lines (interior or exterior based
on bool)
Line bisector(Line l1, Line l2, bool interior) {
    assert(cross(l1.v, l2.v) != 0); // l1 and l2 cannot be parallel!
    T sign = interior ? 1 : -1;
    return {l2.v / abs(l2.v) + l1.v / abs(l1.v) * sign, l2.c / abs(l2.v) +
l1.c / abs(l1.v) * sign};
}

bool intersectll(line a, line b, point &intersect)
{
    ld D = cross(a.n, b.n);
    if (fabs(D) < eps) return 0;
    point p1 = a.n * (a.c / dot(a.n, a.n));
    point d1 = point(-a.n.Y, a.n.X);
    point p2 = b.n * (b.c / dot(b.n, b.n));
    point d2 = point(-b.n.Y, b.n.X);
    ld t = cross(p2 - p1, d2) / cross(d1, d2);
    intersect = p1 + d1 * t;
    return 1;
}

Line normalizeLine(pt a, pt b) {
    int A = b.Y - a.Y;
    int B = b.X - a.X;
    int g = gcd(A, B);
    A /= g;
    B /= g;
    if (A < 0 or A == 0 and B < 0)
        A *= -1, B *= -1;
    int C = cross(pt(B, A), a);
    return Line(A, B, C);
}

// hadnle segment aa
// 00 01 11
//line-ray-segment
// line line 0000 0
// line ray 0001 1
// line seg 0011 3
// ray line 0100 4
// ray ray 0101 5
// ray seg 0111 7
// seg line 1100 12

```

```

// seg ray    1101 13
// seg seg    1111 15
bool intersectll(point a, point b, point c, point d, point &intersect, int
code) {
    ld d1 = cross(a - b, d - c);
    if (lt(abs(d1), 0))
        return false;
    ld t1 = cross(a - c, d - c) / d1;
    ld t2 = cross(a - b, a - c) / d1;
    intersect = a + (b - a) * t1;
    // if ab ray then t1 < -eps
    // if ab seg then t1 < -eps || t1 > 1 + eps    // if ab line then no
checking.. same for cd but using t2
if (code & (1 << 3) and gt(t1, 1)) // seg for 1
return false;
    if (code & (1 << 2) and lt(t1, 0))
        return false;
    if (code & (1 << 1) and gt(t2, 1))
        return false;
    if (code & (1) && lt(t2, 0))
        return false;
    return true;
}

```

Manhattan Trick

Manhattan Trick Reference

Core Transformation:

Map every point (x, y) to (u, v) :

- $u = x + y$
- $v = x - y$

Distance Conversion:

The Manhattan distance between (x_1, y_1) and (x_2, y_2) becomes the Chebyshev (maximum) distance between (u_1, v_1) and (u_2, v_2) :

$$d = \max(|u_1 - u_2|, |v_1 - v_2|)$$

Max Distance in (u, v) :

Instead of checking all pairs, the maximum Manhattan distance in a set of points is simply the largest 1D spread in either the u or v dimension:

$m(m(u) \min(u), m(v) \min(v))$

Generalization

for k dimension space try all possible for + - and the same as 2d space

Others

```
// find a circle of radius r that contains as many points as possible
// O(n^2 log n)
// Small EPS (1e-18) !
pair<int, pt> maximum_circle_cover(vector<pt> p, ld r)
{
    int n = p.size();
    int ans = 0;
    int id = 0;
    ld th = 0;
    for (int i = 0; i < n; ++i)
    {
        // maximum circle cover when the circle goes through this point
        vector<pair<ld, int>> events = {{-pi, +1}, {pi, -1}};
        for (int j = 0; j < n; ++j)
        {
            if (j == i)
                continue;
            ld d = abs(p[i] - p[j]);
            if (d > r * 2)
                continue;
            ld dir = arg(p[j] - p[i]);
            ld ang = acos(d / 2 / r);
            ld st = dir - ang, ed = dir + ang;
            if (st > pi)
                st -= pi * 2;
            if (st <= -pi)
                st += pi * 2;
            if (ed > pi)
                ed -= pi * 2;
            if (ed <= -pi)
                ed += pi * 2;
            events.push_back({st - EPS, +1}); // take care of precisions!
            events.push_back({ed, -1});
            if (st > ed)
            {
                events.push_back({-pi, +1});
                events.push_back({+pi, -1});
            }
        }
    }
}
```

```

    }
    sort(events.begin(), events.end());
    int cnt = 0;
    for (auto &&e : events)
    {
        cnt += e.second;
        if (cnt > ans)
        {
            ans = cnt;
            id = i;
            th = e.first;
        }
    }
}
pt w = pt(p[id].X + r * cos(th), p[id].Y + r * sin(th));
return {ans, w}; // points covered and center of the circle
}

```

// Finds the largest possible radius of a circle that can fit entirely inside a convex polygon using binary search

```

double maximum_inscribed_circle(vector<pt> p)
{
    int n = p.size();
    if (n <= 2)
        return 0;
    double l = 0, r = 20000;
    while (r - l > EPS)
    {
        double mid = (l + r) * 0.5;
        vector<Halfplane> h;
        const int L = 1e9;
        h.push_back(Halfplane(pt(-L, -L), pt(L, -L)));
        h.push_back(Halfplane(pt(L, -L), pt(L, L)));
        h.push_back(Halfplane(pt(L, L), pt(-L, L)));
        h.push_back(Halfplane(pt(-L, L), pt(-L, -L)));
        for (int i = 0; i < n; i++)
        {
            pt z = perp(p[(i + 1) % n] - p[i]);
            z = normalize(z);
            z *= mid;
            pt y = p[i] + z, q = p[(i + 1) % n] + z;
            h.push_back(Halfplane(p[i] + z, p[(i + 1) % n] + z));
        }
        vector<pt> nw = hp_intersect(h);
        if (!nw.empty())
            l = mid;
    }
}

```

```

        else
            r = mid;
    }
    return l;
}

// Calculates the incircle (largest inscribed circle) of a triangle defined
// by points A, B, and C
Circle inCircle(pt A, pt B, pt C) // triangle ABC
{
    T a = abs(B - C);
    T b = abs(C - A);
    T c = abs(A - B);
    pt I = (a * A + b * B + c * C) / (a + b + c); // incenter

    T s = (a + b + c) / 2;
    T area = sqrt(s * (s - a) * (s - b) * (s - c));
    T r = area / s; // inradius

    return Circle(I, r); // center and radius
}

```

```

// handle segment aa
// 00 01 11
//line-ray-segment
// line line 0000 0
// line ray 0001 1
// line seg 0011 3
// ray line 0100 4
// ray ray 0101 5
// ray seg 0111 7
// seg line 1100 12
// seg ray 1101 13
// seg seg 1111 15
bool intersectll(point a, point b, point c, point d, point &intersect, int
code) {
    ld d1 = cross(a - b, d - c);
    if (lt(abs(d1), 0))
        return false;
    ld t1 = cross(a - c, d - c) / d1;
    ld t2 = cross(a - b, a - c) / d1;
    intersect = a + (b - a) * t1;
    // if ab ray then t1 < -eps
    // if ab seg then t1 < -eps || t1 > 1 + eps // if ab line then no
checking.. same for cd but using t2 if (code & (1 <= 3) and gt(t1, 1)) //
seg for 1 // return false; if (code & (1 <= 2) and lt(t1, 0))

```

```

        return false;
    if (code & (1 << 1) and gt(t2, 1))
        return false;
    if (code & (1) && lt(t2, 0))
        return false;
    return true;
}

```

Picks theorem

```

// Calculates the number of integer grid points lying on the line segment AB
int latticeAB(pt A, pt B)
{
    return gcd((int)abs(A.X - B.X), (int)abs(A.Y - B.Y)) + 1;
}

// Applies Pick's Theorem to find the number of integer points inside and on
the boundary of a polygon
// Pick's Theorem -> Area = (interior lattice points) + (boundary lattice
points / 2) - 1
// make T int
pair<int, int> picks(vector<pt> &v) // {inside, boundary}
{
    int n = v.size();
    ld area = 0;
    int boundary = 0;
    for (int i = 0; i < n; ++i)
    {
        area += cross(v[i], v[(i + 1) % n]);
        pt vec = v[(i + 1) % n] - v[i];
        boundary += gcd((int)vec.X, (int)vec.Y); // T has to be int
    }
    int in = (abs(area) - boundary + 2) / 2;
    return make_pair(in, boundary);
}

```

Polygon

```

ld areaTriangle(pt a, pt b, pt c)
{
    return abs(cross(b - a, c - a)) / 2.0;
}

ld perimeter(vector<point > &p) {
    ld ans = 0;

```

```

    int n = p.size();
    for (int i = 0; i < n; i++) ans += dist(p[i] - p[(i + 1) % n]);
    return ans;
}

T areaPolygon(vector<pt> &p)
{
    T area = 0;
    for (int i = 0, n = p.size(); i < n; i++)
    {
        area += cross(p[i], p[(i + 1) % n]); // wrap back to 0 if i == n - 1
    }
    return abs(area) / 2.0;
}

bool above(pt a, pt p)
{
    return p.Y >= a.Y;
}

// check if [PQ] crosses ray from A
bool crossesRay(pt a, pt p, pt q)
{
    return (above(a, q) - above(a, p)) * orient(a, p, q) > 0;
}

// O(N) check to see if point a is inside a general polygon using ray-
// casting
bool inPolygon(vector<pt> p, pt a, bool &onBoundary, bool strict = true) //
// strict false will count border points also
{
    int numCrossings = 0;
    for (int i = 0, n = p.size(); i < n; i++)
    {
        if (onSegment(p[i], p[(i + 1) % n], a))
        {
            onBoundary = true;
            return !strict;
        }
        numCrossings += crossesRay(a, p[i], p[(i + 1) % n]);
    }
    return (numCrossings & 1); // inside if odd number of crossings
}

// checks if a polygon is convex or not
bool isConvex(vector<pt> p)
{
    bool hasPos = false, hasNeg = false;
    int n = p.size();

```

```

for (int i = 0; i < n; ++i)
{
    int sign = orient(p[i], p[(i + 1) % n], p[(i + 2) % n]);
    hasPos |= (sign > 0);
    hasNeg |= (sign < 0);
}
return !(hasPos & hasNeg);
}

// 1 -> point out of polygon
// 0 -> point on border of polygon
// -1 -> point inside polygon
// call prepareConvexPolygon(v) before it
int inPolygonlg(pt pp, vector<pt> &v) // O(log(N))
{
    int n = v.size();
    int o1 = orient(v[0], v[1], pp);
    int o2 = orient(v[0], v[n - 1], pp);
    if (o1 == 0 && onSegment(v[0], v[1], pp))
        return 0;
    else if (o2 == 0 && onSegment(v[0], v[n - 1], pp))
        return 0;
    else if (o1 < 0 || o2 > 0)
        return 1;
    int l = 0, r = n - 1;
    int ans = 1;
    while (l <= r)
    {
        int mid = l + (r - l) / 2;
        int o = orient(v[0], v[mid], pp);
        if (o > 0)
        {
            ans = mid;
            l = mid + 1;
        }
        else
        {
            r = mid - 1;
        }
    }
    int oo = orient(v[ans], v[(ans + 1) % n], pp);
    if (oo > 0)
        return -1;
    else if (oo == 0 && onSegment(v[ans], v[(ans + 1) % n], pp))
        return 0;
    return 1;
}

```

```

}

// minimum distance between 2 polygons
ld polyPoly(vector<pt> p1, vector<pt> p2)
{
    ld ret = 1e18;
    auto lt = [&](pt a, pt b) -> bool
    { return make_pair(a.X, a.Y) < make_pair(b.X, b.Y); };
    auto gt = [&](pt a, pt b) -> bool
    { return make_pair(a.X, a.Y) > make_pair(b.X, b.Y); };
    auto nxt = [&](int a, int sz) -> int
    { return (a + 1) % sz; };
    auto prv = [&](int a, int sz) -> int
    { return (a - 1 + sz) % sz; };
    for (int rep = 0; rep < 2; ++rep)
    {
        swap(p1, p2);
        int ptr1 = 0, ptr2 = 0, n = p1.size(), m = p2.size();
        for (int i = 1; i < n; ++i)
        {
            if (lt(p1[i], p1[ptr1]))
                ptr1 = i;
        }
        for (int i = 1; i < m; ++i)
        {
            if (gt(p2[i], p2[ptr2]))
                ptr2 = i;
        }
        int cnt = 0;
        for (; ptr1 < n; ptr1 = (ptr1 + 1) % n)
        {
            if (cnt == n)
                break;
            cnt++;
            pt base = p1[nxt(ptr1, n)] - p1[ptr1];
            Line l(p1[nxt(ptr1, n)], p1[ptr1]);
            while (sgn(cross(base, p2[nxt(ptr2, m)] - p2[ptr2])) == sgn(cross(base,
p2[ptr2] - p2[prv(ptr2, m)])))
            {
                ptr2 = nxt(ptr2, m);
            }
            ret = min(ret, segSeg(p1[nxt(ptr1, n)], p1[ptr1], p2[nxt(ptr2, m)],
p2[ptr2]));
            ret = min(ret, segSeg(p1[nxt(ptr1, n)], p1[ptr1], p2[ptr2], p2[prv(ptr2,
m)]));
        }
    }
}

```

```

    }
    return ret;
}

point polygonCenteriod(vector<point > points) {
    ld x = 0, y = 0, a = 0, c;
    for (int i = 0; i < points.size(); i++) {
        int j = (i + 1) % points.size();
        c = cross(points[i], points[j]), a += c;
        x += (points[i].X + points[j].X) * c;
        y += (points[i].Y + points[j].Y) * c;
    }
    if (dcmp(a, 0) == 0) return (points[0] + points.back()) * 0.5l;
    a /= 2, x /= 6 * a, y /= 6 * a;
    return point(x, y);
}

```

Rays

```

// Calculates the shortest distance between point p and a ray starting at a
// and passing through b
int pointRay(pt p, pt a, pt b) // a is ray's origin, b defines direction
{
    pt dir = b - a; // ray direction vector
    pt v = p - a;    // vector from origin to point

    T u = dot(v, dir);

    if (u <= 0) // Point is behind the ray origin
    {
        return abs(v); // distance from p to a
    }
    else // Point is in front of the ray
    {
        // Compute closest point on the ray: a + (u / ||dir||^2) * dir
        pt closest = a + (u / norm(dir)) * dir;
        return abs(p - closest); // distance from p to closest point
    }
}

// Calculates the minimum distance between two rays (ray ab and ray cd)
ld rayRay(pt a, pt b, pt c, pt d)
{
    T inf = 1e12;

```

```

    b = a + (b - a) * inf;
    d = c + (d - c) * inf;
    if (inters(a, b, c, d).size())
        return 0;
    pt dummy;
    return min({segPoint(a, b, c, dummy), segPoint(a, b, d, dummy),
segPoint(c, d, a, dummy), segPoint(c, d, b, dummy)});
}

```

Rectangle union

```

#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;
template<typename T>
using orderedset = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;
#define int long long
#define ll long long
#define ld long double
#define endl '\n'
#define wady \
    ios_base::sync_with_stdio(0); \
    cin.tie(0); \
    cout.tie(0);

void files() {
#ifdef ONLINE_JUDGE
    freopen("in.txt", "r", stdin);
    freopen("out.txt", "w", stdout);
#endif
}

const int N = 2e5 + 10;

struct Event {
    int x, y1, y2, type;
    bool operator<(const Event &other) const {
        if (x != other.x) return x < other.x;
        return type > other.type;
    }
};

```

```

int cnt[N * 8];
int len[N * 8];
int y_coords[N];

void update_node(int node, int tl, int tr) {
    if (cnt[node] > 0) {
        len[node] = y_coords[tr + 1] - y_coords[tl];
    } else if (tl != tr) {
        len[node] = len[node << 1] + len[node << 1 | 1];
    } else {
        len[node] = 0;
    }
}

void update(int node, int tl, int tr, int ql, int qr, int val) {
    if (ql > tr || qr < tl) return;
    if (ql <= tl && tr <= qr) {
        cnt[node] += val;
        update_node(node, tl, tr);
        return;
    }
    int mid = (tl + tr) >> 1;
    update(node << 1, tl, mid, ql, qr, val);
    update(node << 1 | 1, mid + 1, tr, ql, qr, val);
    update_node(node, tl, tr);
}

void solve(int tc) {
    int n;
    cin >> n;
    vector<Event> events;
    vector<int> ys;
    for (int i = 0; i < n; i++) {
        int x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        if (x1 > x2) swap(x1, x2);
        if (y1 > y2) swap(y1, y2);
        events.push_back({x1, y1, y2, 1});
        events.push_back({x2, y1, y2, -1});
        ys.push_back(y1);
        ys.push_back(y2);
    }
    sort(events.begin(), events.end());
    sort(ys.begin(), ys.end());
    ys.erase(unique(ys.begin(), ys.end()), ys.end());
}

```

```

    for (int i = 0; i < ys.size(); i++) y_coords[i] = ys[i];
    int m = ys.size();
    fill(cnt, cnt + 8 * m, 0);
    fill(len, len + 8 * m, 0);
    int ans = 0;
    for (int i = 0; i < events.size(); i++) {
        if (i > 0) ans += len[1] * (events[i].x - events[i - 1].x);
        int ql = lower_bound(ys.begin(), ys.end(), events[i].y1) - ys.begin();
        int qr = lower_bound(ys.begin(), ys.end(), events[i].y2) - ys.begin()
- 1;
        if (ql <= qr) update(1, 0, m - 2, ql, qr, events[i].type);
    }
    cout << ans << endl;
}

signed main() {
    wady
    int t = 1;
    int tc = 1;
    while (t--)
        solve(tc++);
}

```

Rotating Calipers

```

// Computes max distance between any two points (diameter) of a convex polygon
ld farthest_pair(vector<pt> &p)
{
    vector<pt> hull = convex_hull(p);
    int m = hull.size();
    int j = 1;
    ld d = 0;
    // rotating calipers
    for (int i = 0; i < m; ++i)
    {
        while (norm(hull[i] - hull[(j + 1) % m]) > norm(hull[i] - hull[j]))
            j = (j + 1) % m;
        d = max((T)d, norm(hull[i] - hull[j]));
    }
    return sqrtl(d);
}

// Computes min width of a parallel strip that can enclose all points
ld min_width(vector<pt> p)

```

```

{
    vector<pt> hull = convex_hull(p);
    int m = hull.size();
    if (m <= 2)
        return 0;
    ld ret = 1e18;
    // rotating calipers
    for (int i = 0, j = 1; i < m; ++i)
    {
        pt u = hull[i];
        pt v = hull[(i + 1) % m];
        while (areaTriangle(u, v, hull[(j + 1) % m]) > areaTriangle(u, v,
hull[j]))
            j = (j + 1) % m;
        ld height = 2 * areaTriangle(u, v, hull[j]) / dist(u, v);
        ret = min(ret, height);
    }
    return ret;
}

```

```

// area and perimeter of minimum enclosing rectangle
pair<ld, ld> minimum_enclosing_rectangle(vector<pt> &p)
{

```

```

    int n = p.size();
    if (n <= 2)
    {
        ld d = 0;
        for (int i = 0; i < n; ++i)
            d += dist(p[i], p[(i + 1) % n]);
        return {0, d};
    }
    int mndot = 0;
    double tmp = dot(p[1] - p[0], p[0]);
    for (int i = 1; i < n; i++)
    {
        if (dot(p[1] - p[0], p[i]) <= tmp)
        {
            tmp = dot(p[1] - p[0], p[i]);
            mndot = i;
        }
    }
    ld perim = 1e18, area = 1e18;
    int i = 0, j = 1, mxdot = 1;
    while (i < n)
    {
        pt cur = p[(i + 1) % n] - p[i];

```

```

while (cross(cur, p[(j + 1) % n] - p[j]) >= 0)
    j = (j + 1) % n;
while (dot(p[(mxdot + 1) % n], cur) >= dot(p[mxdot], cur))
    mxdot = (mxdot + 1) % n;
while (dot(p[(mndot + 1) % n], cur) <= dot(p[mndot], cur))
    mndot = (mndot + 1) % n;
Line l(p[i], p[(i + 1) % n]);
ld w = (dot(p[mxdot], cur) / abs(cur) - dot(p[mndot], cur) / abs(cur));
ld h = l.dist(p[j]);
perim = min(perim, 2.0 * (w + h));
area = min(area, w * h);
i++;
}
return {area, perim};
}

```

Segment

```

bool inDisk(pt a, pt b, pt p)
{
    return dot(a - p, b - p) <= EPS;
}

bool onSegment(pt a, pt b, pt c) // check if c on ab segment
{
    return orient(a, b, c) == 0 && inDisk(a, b, c);
}

// Finds the intersection of segments ab and cd
bool properInter(pt a, pt b, pt c, pt d, pt &out)
{
    T oa = orient(c, d, a),
      ob = orient(c, d, b),
      oc = orient(a, b, c),
      od = orient(a, b, d);
    // Proper intersection exists iff opposite signs
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
    {
        out = (a * ob - b * oa) / (ob - oa);
        return true;
    }
    return false;
}

// Returns a set of all intersection points between segments ab and cd
set<pair<ld, ld>> inters(pt a, pt b, pt c, pt d)
{

```

```

set<pair<ld, ld>> s;
pt out;
if (a == c || a == d)
{
    s.insert(make_pair(a.X, a.Y));
}
if (b == c || b == d)
{
    s.insert(make_pair(b.X, b.Y));
}
if (s.size())
    return s;

if (properInter(a, b, c, d, out))
    return {make_pair(out.X, out.Y)};
if (onSegment(c, d, a))
    s.insert(make_pair(a.X, a.Y));
if (onSegment(c, d, b))
    s.insert(make_pair(b.X, b.Y));
if (onSegment(a, b, c))
    s.insert(make_pair(c.X, c.Y));
if (onSegment(a, b, d))
    s.insert(make_pair(d.X, d.Y));

return s;
}

// Calculates the shortest distance from point p to segment ab and
// identifies the closest point 'out
ld segPoint(pt a, pt b, pt p, pt &out)
{
    if (a != b)
    {
        Line l(a, b);
        if (l.cmpProj(a, p) and l.cmpProj(p, b)) // if closest to projection
(in between a & b)
        {
            out = l.proj(p);
            return l.dist(p); // output distance to line
        }
    }
    if (abs(p - a) - abs(p - b) > EPS)
        out = b;
    else
        out = a;
    return min(abs(p - a), abs(p - b)); // otherwise distance to A or B
}

```

```

// Calculates the minimum distance between two segments ab and cd
ld segSeg(pt a, pt b, pt c, pt d)
{
    pt dummy;
    if (properInter(a, b, c, d, dummy))
        return 0;
    return min({segPoint(a, b, c, dummy), segPoint(a, b, d, dummy),
                segPoint(c, d, a, dummy), segPoint(c, d, b, dummy)});
}

```

Transformations

```

// Shifts point p by the displacement vector v
pt translate(pt v, pt p) { return p + v; }

// Scales point p relative to a center c by a given factor
pt scale(pt c, T factor, pt p) { return c + (p - c) * factor; }

// Rotates point p around center c by an angle a (in radians)
pt rot(pt p, pt c, ld a) { return c + pt(cos(a), sin(a)) * (p - c); }

// Returns a unit vector in the same direction as vector a
pt normalize(pt a) { return a / abs(a); }

// Performs a similarity transformation mapping segment pq to segment fp-fq,
// applied to point r
pt linearTransfo(pt p, pt q, pt r, pt fp, pt fq) { return fp + (r - p) * (fq - fp) / (q - p); }

```

Winding number

```

/*
the winding number of a closed curve in the plane around a given point
is an integer representing the total number of times that the curve travels
counterclockwise around the point
*/
// inf -> a is on edge of p
// 0 -> a is outside p
// otherwise -> a is inside p
int winding_number(vector<pt> &p, pt a)
{
    int ret = 0;
    for (int i = 0, n = p.size(); i < n; ++i)
    {

```

```

    pt cur = p[i];
    pt nxt = p[(i + 1) % n];
    if (onSegment(cur, nxt, a))
        return inf;
    if (cur.Y <= a.Y and nxt.Y > a.Y and cross(nxt - cur, a - cur) > 0)
        ret++;
    else if (cur.Y > a.Y and nxt.Y <= a.Y and cross(nxt - cur, a - cur) <
0)
        ret--;
    }
    return ret;
}

```